Reducing Communication Costs in Decentralized Federated Learning using Torrents

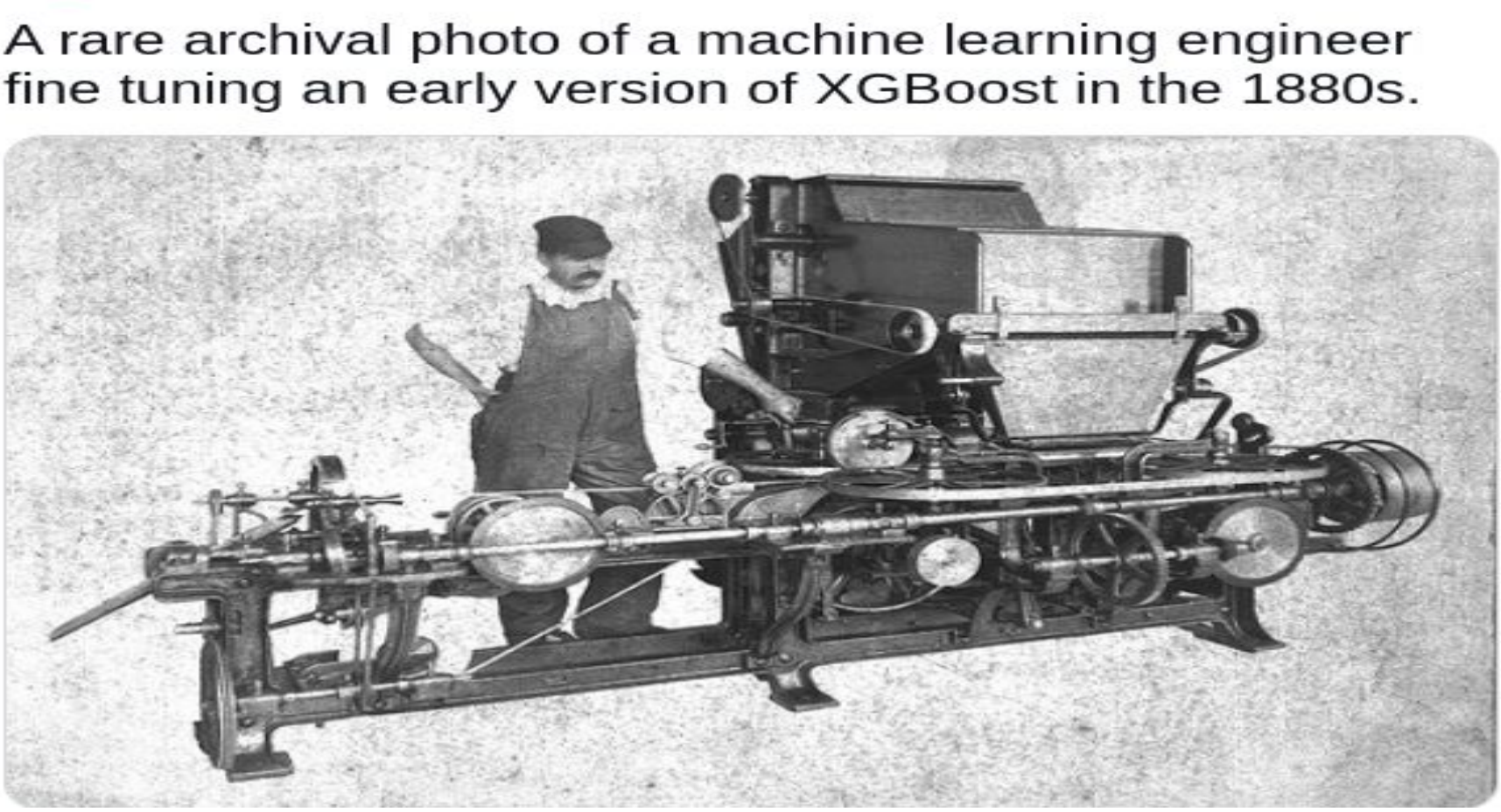
Ender Minyard, Steven Kolawole

Presenting @ ML Efficiency Workshop, Deep Learning Indaba ‘22





Once upon a time...

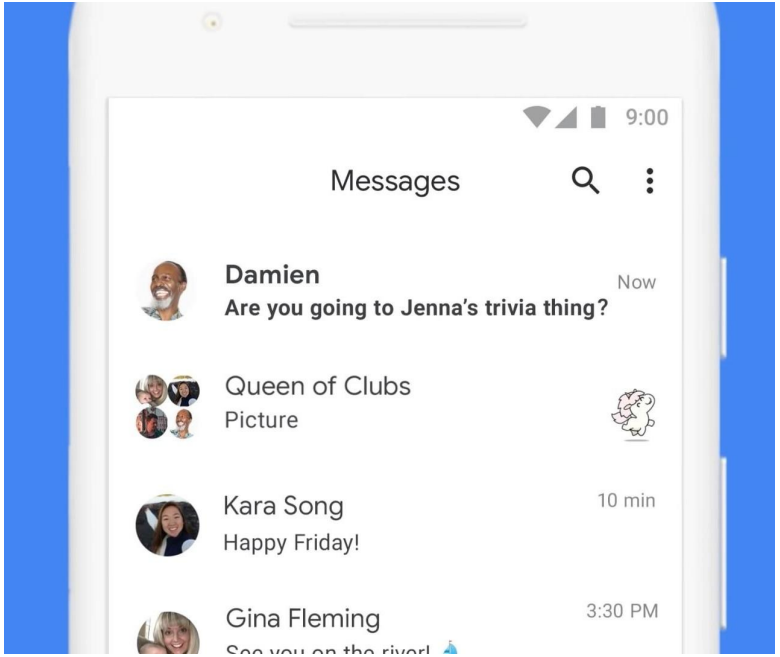


Federated learning: private ML for the edge 📱

Federated learning allows training data to stay on edge devices. These edge devices are used to train a model. Data stays local.

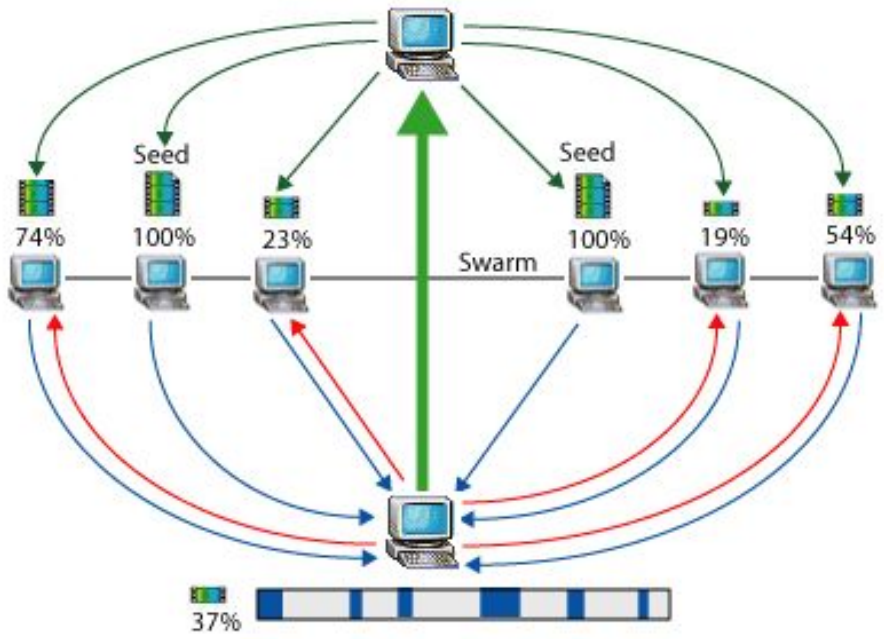
Training data is not sent to a central server.

Federated learning is used in Siri, Android messages and the Gboard mobile keyboard.



Decentralized Mutable Torrents

- Instead of downloading files to a central server, torrenting involves downloading files from other users' devices on the network. Conversely, users upload files from their own devices for other users to download.
- Torrenting is the most popular form of peer-to-peer (P2P) file-sharing. A peer can download a file from several other users at once, or upload a file to multiple other users simultaneously.
- **Updating Torrents Via DHT Mutable Items:** Allows publishers to serve content that might change over time in a more decentralized fashion.



The training process 

1. Client selection

The server samples from a set of clients meeting eligibility requirements. For example, mobile phones might only check in to the server if they are plugged in, on an unmetered wi-fi connection, and idle, in order to avoid impacting the user of the device.

From Advances and Open Problems in Federated Learning (2021)

2. Broadcast

The selected clients download the current model weights and a training program from a mutable torrent.

This torrent is a shared file saved on the client’s file system.

3. Client computation

Each selected device locally computes an update to the model by executing the training program, which might for example run SGD on the local data (as in Federated Averaging).

From Advances and Open Problems in Federated Learning (2021)

4. Aggregation

Each edge device (client) saves their model update to the torrent’s distributed hash table. The torrent is already being seeded locally.

5. Model update

One peer computes the shared model based on the aggregated update computed from the clients that participated in the current round.

This shared model becomes the content of the mutable torrent.

Our contribution to the field

In (Caldas et al., 2018), the authors show that lossy compression (of the models downloaded by clients) and Federated Dropout (exchanging smaller sub-models between client and server instead of the global model) reduce communication costs without degrading the quality of the final model.

Reducing the impact of federated learning on client resources allows higher capacity models to be trained.

Our work allows higher capacity models to be trained by using mutable torrents to aggregate local updates from clients.

Current Step

- Python implementation of DHT Mutable Torrents (https://github.com/lmatteis/bittorrent.org/blob/master/beps/bep_0046.rst)

What we need

- A collaborator/mentor with practical knowledge of Federated Learning
- **Nice-to-have:** A collaborator familiar with torrents design